USB Device Control & Monitoring Framework

Project Documentation · Oct 14, 2026 · @Anjali

Abstract

This project delivers a working USB Device Control & Monitoring Framework for Linux that detects USB devices in real time, blocks unauthorized, cloned and BadUSB devices automatically, audits file transfers to USB drives, and produces an HTML security audit report. It was built in Python on Kali Linux and tested with real hardware: a USB mouse and a USB pen drive.

The framework listens to kernel udev events, fingerprints each device (Vendor ID, Product ID, serial number, manufacturer, product and interface class), and applies an allowlist/blocklist policy with default deny. Unauthorized devices are blocked through the kernel's USB authorization interface. On approved drives, every file created, written, renamed or deleted is logged with its size, user and SHA-256 hash, and 10 or more files copied within 60 seconds raise a large-transfer alert.

All tests passed: approved devices were allowed, unknown and blocklisted devices were blocked by the kernel, a serial-mismatch clone was detected, and simulated BadUSB, attack-board and fake-serial devices were caught. The source code, logs, screenshots and report are published at github.com/anjalicheriath003/usb-device-control-monitoring.

1. Introduction and motivation

USB ports are one of the easiest ways to attack a computer or steal data from it, because operating systems trust USB devices automatically. A device is enumerated, given drivers and made usable within a second of being plugged in, with no question asked.

The main USB-based threats this project addresses are:

Threat	What happens	Example
Data exfiltration	An insider copies confidential files to a pen drive	Customer records copied before leaving a company
Malware introduction	An infected drive brings malware into the network	USB worms that spread through autorun or user clicks
BadUSB / keystroke injection	A device pretends to be a keyboard and types commands faster than a human	Rubber Ducky, Digispark, Teensy boards

Threat	What happens	Example
Device spoofing	A device copies the IDs of an approved device to get through a filter	A cheap drive reprogrammed with a trusted Vendor ID and Product ID
Unauthorized hardware	Personal or unknown devices connected to work machines	A personal pen drive used on an office PC

Monitoring and controlling USB activity lets a security team detect data-theft attempts, stop malicious devices before they act, and keep an audit trail of what was connected, when, and what was copied.

2. Objectives and scope

All five objectives of the project were met:

#	Objective	Status
1	Detect USB devices when they are plugged in and unplugged	Achieved
2	Maintain an allowlist and blocklist of approved USB devices	Achieved
3	Automatically block suspicious or unauthorized USB devices	Achieved
4	Track file transfers between the system and USB devices	Achieved
5	Generate detailed USB activity and audit reports	Achieved

In scope:

- Unauthorized USB detection: identifying newly connected devices, comparing their IDs against the allowlist and blocklist, and detecting spoofing attempts.
- Auto-blocking: denying access to unapproved devices and logging every attempt.
- Optionally disabling USB storage: shown manually by de-authorizing a device through sysfs.
- File movement auditing: logging create, write, rename and delete operations on USB drives, by whom and when, and detecting large or abnormal transfers.
- Reporting and alerting: timestamped logs of all events and a final HTML security report.

Out of scope: Windows support, network-wide (multi-machine) management, and tracking files copied from a USB drive onto the computer.

3. Tools and technologies

The framework is written in Python 3 and runs on Kali Linux inside VirtualBox.

Tool / technology	Purpose in this project
Kali Linux (Rolling, kernel 6.16.8)	Operating system the framework runs on
Oracle VirtualBox	Virtual machine host; USB devices are passed to Kali via Devices → USB
Python 3.13	Programming language for all modules
pyudev	Listens to kernel udev events for USB connect / disconnect in real time
watchdog (inotify)	Watches a mounted USB drive for file create, write, rename and delete events
hashlib (SHA-256)	Fingerprints the contents of each file written to USB
json	Stores the security policy and writes the event log (one JSON record per line)
Linux sysfs (/sys/bus/usb/devices)	Source of device details and the `authorized` file used to block devices
lsusb, lsblk, udevadm	Command-line tools used to explore and verify USB behaviour
Firefox	Viewing the HTML audit report
Git and GitHub	Version control (one commit per stage) and publishing the project
Test hardware	YSPRINGTECH USB optical mouse (10c4:0005) and a DIGITEK 29.3 GB pen drive (048d:1234)

4. Background: how USB devices communicate with Linux

The kernel announces every USB device, exposes its details as files, and lets root decide whether drivers may use it. The framework is built on these three facts, all verified by hand in the first stage of the project.

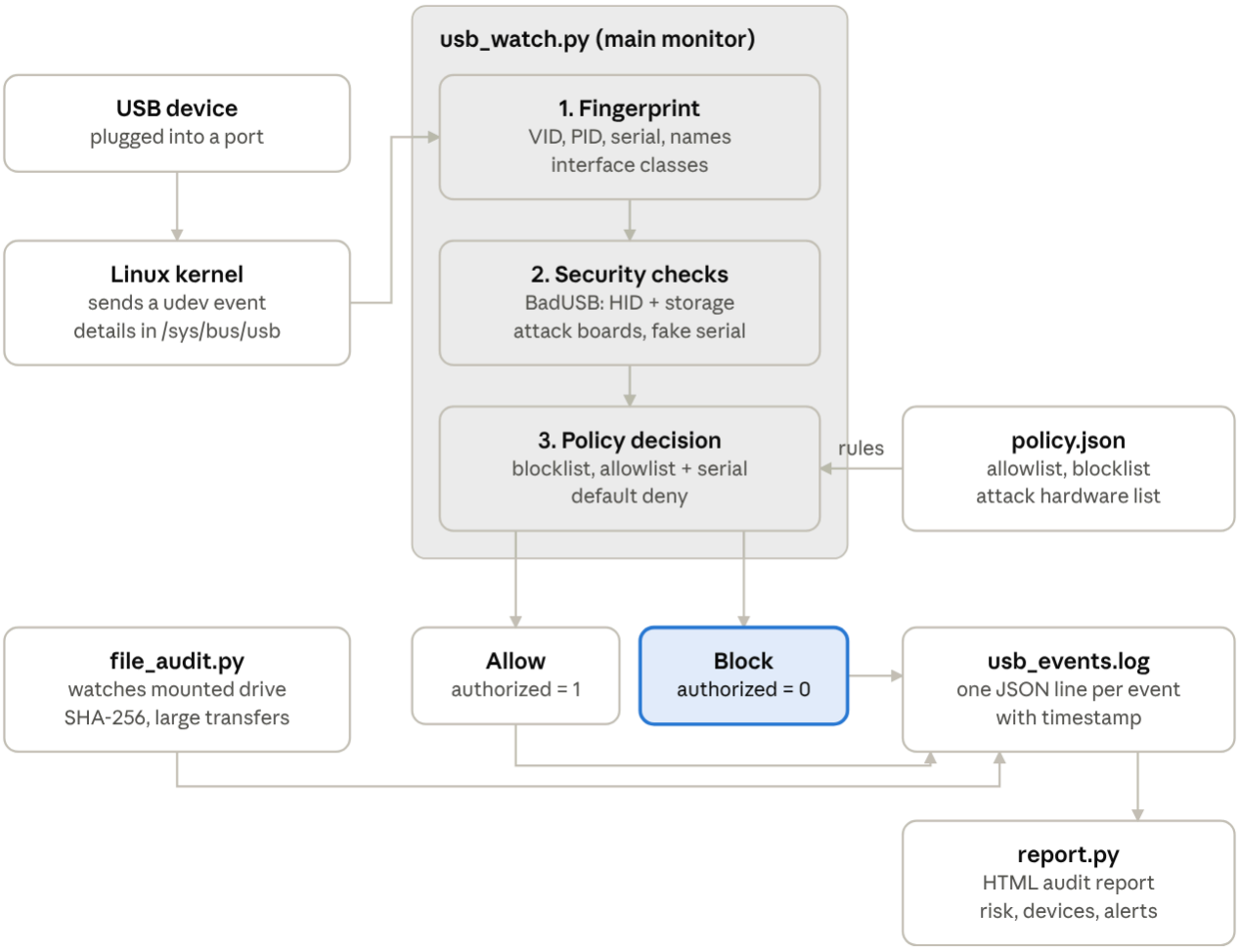
1. **Enumeration and udev events.** When a device is plugged in, the kernel reads its descriptors and sends a uevent. `udevadm monitor` showed the sequence `add → bind` on connect and `unbind → remove` on disconnect. `1-2` is the device (its port) and `1-2:1.0` is its interface (what it does).
2. **sysfs.** Each device appears as a folder under `/sys/bus/usb/devices/<port>/`, with files such as `idVendor`, `idProduct`, `manufacturer`, `product` and `serial`. The test mouse reported `10c4`, `0005`, `YSPRINGTECH`, `USB OPTICAL MOUSE`, and had no `serial` file.
3. **Interface classes.** `bInterfaceClass` tells what a device does: `03` = HID (keyboard or mouse), `08` = mass storage. A device that reports both is the BadUSB pattern.
4. **The authorized flag.** Writing `0` to `/sys/bus/usb/devices/<port>/authorized` makes the kernel detach every driver. In the manual test the mouse stopped working for exactly 10 seconds (`unbind` at 1452.99 s, `bind` at 1463.05 s) while still being listed by `lsusb`: blocked, not unplugged.

Two observations shaped the security design:

- **Device names can lie.** `lsusb` named the mouse's chip maker as Silicon Labs (looked up from Vendor ID `10c4`), but the device itself reported YSPRINGTECH; the pen drive called itself just "USB Disk 2.0". Names are self-reported and cannot be trusted alone.
- **Device numbers and ports change.** The mouse moved from Device 003 to 004 after reconnecting, and the pen drive took port `1-2` once the mouse was removed. Identity must come from VID, PID and serial, never from the port.

5. System architecture

The framework has three programs that share one policy file and one event log.



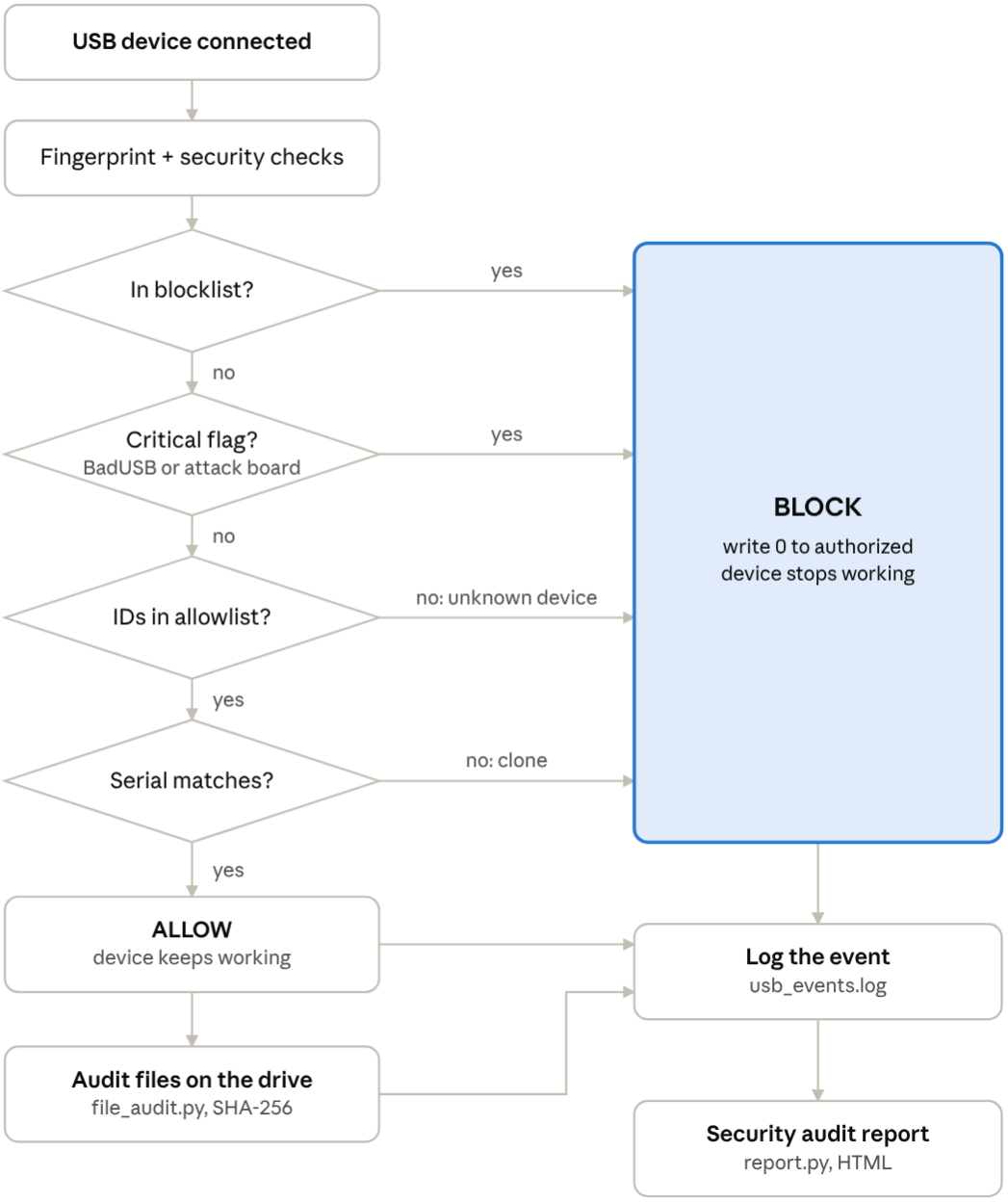
framework architecture · 3 checks, 2 outcomes, 1 log

The kernel reports each new device to `usb_watch.py`, which fingerprints it, runs the security checks and decides using `policy.json`; a blocked device has `0` written to its `authorized` file. `file_audit.py` records file activity on approved drives, and `report.py` turns the shared log into the HTML audit report.

6. Workflow and flowchart

Each device goes through four questions in a fixed order, and a single failed check is enough to block it.

A device is allowed only if it passes all four checks



decision flowchart · 4 checks, 2 outcomes

Only a device that is not blocklisted, has no critical flag, is in the allowlist and has the approved serial is allowed; every decision, and every file event on an allowed drive, is written to the log that feeds the report.

7. Implementation

The framework was built in ten stages, each tested and committed to Git before the next, so the GitHub history shows how it grew.

Stage	What was built	Result
1	Manual exploration with lsusb, udevadm, sysfs and the authorized file	USB events, fingerprint fields and blocking understood
2	First pyudev script printing connect / remove events	Real-time detection working
3	Reading VID, PID, manufacturer, product, serial	Device fingerprinting; missing serial handled as "(none)"
4	policy.json allowlist / blocklist and a decision function	ALLOWED / BLOCKED with default deny
5	Writing 0 to the authorized file for BLOCKED devices	Automatic blocking by the kernel
6	Timestamped JSON event log	Permanent audit trail
7	file_audit.py with watchdog and SHA-256	File auditing and large-transfer alert
8	Serial check, interface classes, attack-hardware list, fake-serial check, tests	Clone and BadUSB detection
9	report.py	HTML security audit report
10	README and GitHub publishing	Project published

Project files

File	Role
`usb_watch.py`	Main monitor: listens for USB events, fingerprints devices, runs security checks, decides, blocks, logs
`policy.json`	Allowlist, blocklist and known attack hardware
`file_audit.py`	Audits all file activity on a mounted USB drive
`report.py`	Builds `reports/usb_audit_report.html` from the log
`test_detection.py`	Runs simulated attack devices through the real detection functions
`logs/usb_events.log`	Audit trail, one JSON record per event

usb_watch.py: main functions

- `read_attr()` reads one sysfs attribute and returns "(none)" if it is missing, so devices without a serial do not crash the monitor.
- `get_interface_classes()` reads `bInterfaceClass` from every interface folder, retrying for up to one second because interfaces appear just after the device.
- `check_suspicious()` returns a list of (severity, message) flags for BadUSB, attack hardware and fake serials.
- `decide()` applies the policy in a fixed order and returns the decision and its reason.
- `block_device()` writes `0` to the device's `authorized` file and reports clearly if root rights are missing.
- `log_event()` appends a timestamped JSON record to the log.
- `main()` runs the monitor; the `if __name__ == "__main__"` guard lets the test script import the functions without starting it.

file_audit.py

A watchdog event handler class reacts to file events on the drive. `on_closed` fires when a file is fully written, so its size and SHA-256 hash are final. A sliding 60-second window counts writes and raises `LARGE_TRANSFER` at 10 files. On exit it logs a `TRANSFER_SUMMARY` with the total files and bytes.

report.py

Reads every log line back into a dictionary (skipping damaged lines), counts events, groups them per device by VID, PID and serial, assigns a risk level, and writes an HTML page with five sections. All device-supplied text is HTML-escaped, so a malicious device name cannot inject code into the report.

Problems solved during development

Problem	Cause	Fix
Mouse and pen drive not visible in Kali	VM does not see host USB devices	Passed through via VirtualBox Devices → USB
`ModuleNotFoundError: pyudev`	Library not installed	`sudo apt install python3-pyudev`
`IndentationError` on line 1	A space copied before `import json`	Removed the leading space

Problem	Cause	Fix
Log timestamps 9.5 hours behind	Kali set to US Eastern time	`timedatectl set-timezone Asia/Kolkata`
`Permission denied` on the log file	File created by root via sudo	`chown` the folder; `os.chmod(LOG_FILE, 0o666)` in `log_event()`
Pen drive not found on port 1-3	Port changed after the mouse was removed	Found the port by Vendor ID with grep

8. Security techniques

The framework layers nine blue-team techniques, so a device that defeats one check is still caught by another.

Technique	How it works	Threat it counters
Hardware event monitoring	pyudev receives every USB add / remove event from the kernel	Unnoticed connection of any device
Device fingerprinting	VID + PID + serial + manufacturer + product + interface classes	Identifying exactly which device is connected
Allowlist / blocklist with default deny	Blocklist first, then allowlist; anything unknown is blocked	Personal and unknown devices
Clone detection	IDs match an approved rule but the serial differs → BLOCKED, "SPOOF SUSPECTED"	Devices copying a trusted device's IDs
BadUSB detection	Interface classes include both `03` (HID) and `08` (storage) → CRITICAL, always blocked	Keystroke-injection devices disguised as drives
Known attack hardware	VID:PID list of Digispark, Teensy, Arduino Leonardo, SparkFun Pro Micro → CRITICAL	Common BadUSB boards
Fake-serial and no-serial warnings	Serials such as `000000000000` or `123456789`, or storage with no serial → WARNING	Cheap clones that cannot be uniquely identified

Technique	How it works	Threat it counters
File auditing with SHA-256	Every file written to USB is logged with size, user, time and hash	Proving what data left the machine
Large-transfer detection	10+ files written within 60 seconds → alert	Bulk data exfiltration

Decision order

`decide()` applies the rules in this order; the first match wins:

1. Device is in the **blocklist** → BLOCKED.
2. Any **CRITICAL** flag (BadUSB pattern or known attack hardware) → BLOCKED, even if the device is allowlisted.
3. IDs and serial match an **allowlist** rule → ALLOWED.
4. IDs match an allowlist rule but the **serial does not** → BLOCKED as spoofing.
5. Anything else → BLOCKED (**default deny**).

CRITICAL checks run before the allowlist on purpose. A BadUSB device that copies an approved drive's IDs and serial perfectly passes every identity check, but its hidden keyboard interface still gets it blocked.

9. Testing and results

All 16 test cases passed. Tests 1–10 used real hardware in Kali; tests 11–16 fed simulated devices into the same detection functions through `test_detection.py`, because no attack hardware was available.

Real-hardware tests

#	Test	Expected	Observed
1	Mouse unplugged / re-plugged with `udevadm monitor` running	add / bind and unbind / remove events	`unbind → remove → add → bind` on port 1-2
2	Manual block: `echo 0 > authorized` for 10 s	Mouse stops, then recovers	Mouse frozen 10.06 s; still listed by lsusb
3	Mouse connected, mouse in allowlist	ALLOWED	ALLOWED – in allowlist: My YSPRINGTECH mouse

#	Test	Expected	Observed
4	Mouse connected, removed from allowlist	BLOCKED, default deny	BLOCKED – not in allowlist (unknown device)
5	Mouse in both blocklist and allowlist	BLOCKED, blocklist wins	BLOCKED – in blocklist: Banned mouse (test)
6	Unknown mouse with auto-blocking enabled	Kernel blocks it	"device blocked by the kernel"; `authorized` = 0; pointer stopped
7	Event logging	Every event saved with correct time	5 JSON records per session, times matching the system clock
8	15 files written to the pen drive	Each logged with size and SHA-256; alert at 10	15 FILE_WRITTEN events, LARGE_TRANSFER at file 10
9	Rename and delete on the pen drive	Logged	FILE RENAMED record_1 → renamed; FILE DELETED record_2
10	Approved serial changed to simulate a clone	BLOCKED as spoof	SPOOF SUSPECTED: serial … 26583 is not the approved … 26999; device blocked

Simulated attack tests (`test_detection.py`)

#	Simulated device	Expected	Observed
11	Real pen drive (048d:1234, correct serial, storage)	ALLOWED	ALLOWED
12	Real mouse (10c4:0005, HID)	ALLOWED	ALLOWED
13	Perfect copy of the pen drive + hidden keyboard	BLOCKED, CRITICAL	BLOCKED – BadUSB pattern: HID + storage
14	Digispark board (16d0:0753)	BLOCKED, CRITICAL	BLOCKED – Known attack hardware: Digispark ATtiny85
15	Unknown drive with serial 000000000000	WARNING + BLOCKED	WARNING fake serial; BLOCKED – unknown device
16	Pen drive IDs with a different serial	BLOCKED as spoof	BLOCKED – SPOOF SUSPECTED

Audit report results

The generated report covered 55 events from 9 to 14 October 2026: 2 unique devices, 3 allowed connections, 1 blocked connection, 15 files (441 bytes) written to USB and 1 large-transfer alert, giving an overall risk of **HIGH**.

10. Sample outputs and screenshots

The outputs below are copied from the real test runs.

Clone (serial spoof) detected and blocked:

```
USB CONNECTED: 1-2
   Vendor ID    : 048d
   Product ID   : 1234
   Manufacturer : USB
   Product      : Disk 2.0
   Serial       : 0718361110626126583
   DECISION     : BLOCKED - SPOOF SUSPECTED: IDs match 'My DIGITEK pen drive'
but serial 0718361110626126583 is not the approved serial 0718361110626126999
   ACTION       : device blocked by the kernel
```

File auditing with a large-transfer alert:

```
FILE CREATED : audit_test/record_10.txt
FILE WRITTEN : audit_test/record_10.txt  (30 bytes)
sha256=1336be74e5c2d4e2...

*** ALERT: LARGE TRANSFER - 10 files copied to USB in under 60s by 'kali' -
possible data theft! ***

FILE RENAMED : audit_test/record_1.txt -> audit_test/renamed.txt
FILE DELETED : audit_test/record_2.txt
SUMMARY: 15 files written to USB (441 bytes) by 'kali'
```

Log record (one JSON line per event):

```
{"time": "2026-10-09 22:54:10", "event": "DEVICE_ALLOWED", "reason": "in
allowlist: My YSPRINGTECH mouse", "port": "1-2", "vendor_id": "10c4",
"product_id": "0005", "manufacturer": "YSPRINGTECH", "product": "USB OPTICAL
MOUSE", "serial": "(none)"}
```

Screenshot index

All screenshots are in `docs/screenshots/` of the GitHub repository; insert them in the exported document as figures.

Figure	File	Shows
1	02_lsusb_after.png	Mouse visible in Kali after USB pass-through
2	03_udevadm_monitor.png	Kernel add / bind / unbind / remove events
3	04_sysfs_details.png	Device details read from sysfs
4	05a_block_events.png, 05b_block_command.png	Manual block for 10 seconds
5	05_block_test.png	Block test with command and events together
6	07_fingerprint.png	Device fingerprint printed by the monitor
7	08_decision_allowed.png	Allowlisted device ALLOWED
8	09_decision_unknown.png	Unknown device BLOCKED (default deny)

Figure	File	Shows
9	10_decision_blocklist.png	Blocklist overrides allowlist
10	11_auto_block.png	Automatic kernel blocking, authorized = 0
11	12_log_file.png	Timestamped event log
12	13_file_audit.png	File auditing and LARGE TRANSFER alert
13	14_spoof_serial.png	Clone detection by serial mismatch
14	15_badusb_tests.png	Simulated BadUSB and attack-hardware tests
15	16_report_full.png	HTML security audit report

11. Limitations and future enhancements

The framework works reliably for its scope; its main limits come from running reactively on a single Linux machine.

Limitation	Possible enhancement
Blocking is reactive: a device is usable for a fraction of a second before it is blocked	Strict mode: set `authorized_default` to 0 so every new device starts blocked until approved
Linux only (sysfs and udev)	A Windows version using WMI / PowerShell `Get-PnpDevice`
A device that copies every detail of an approved device, including its serial, is caught only if its interfaces differ	Combine with physical port control and endpoint security policies
Files copied from USB to the computer are not tracked	Hash-index the drive and watch system folders for matching files
`usb_watch.py` and `file_audit.py` run separately; the drive is mounted by hand	Detect block devices and start auditing automatically on mount
Policy is edited by hand in JSON	An `enroll` command that adds a connected device to the allowlist
Alerts appear only in the terminal and the log	Desktop notifications and email or SIEM forwarding

Limitation	Possible enhancement
Log file is world-writable (chmod 666) for simplicity	Root-owned, append-only log with integrity checks
The monitor is started manually	Run as a systemd service at boot

12. Learning outcomes and conclusion

The project produced a working USB security framework that detects, fingerprints, controls and audits USB devices, and every objective was met and tested on real hardware.

Learning outcomes:

- How USB hardware communicates with Linux: enumeration, udev events, sysfs, interface classes and kernel authorization.
- How attackers misuse USB: data theft, malware delivery, BadUSB keystroke injection and device spoofing.
- Why device identity must combine several fields, since names are self-reported and ports and device numbers change.
- How to enforce an allowlist / blocklist with default deny, and why critical checks must run before trust decisions.
- File auditing with inotify and SHA-256 hashing, and detecting abnormal transfers with a sliding time window.
- Python skills: functions, dictionaries, classes and event handlers, exception handling, JSON, unit-style testing.
- Practical tooling: Kali Linux, VirtualBox USB pass-through, Linux permissions and time zones, Git and GitHub.

Conclusion: USB ports remain an easy path for data theft and malware because operating systems trust devices automatically. This framework replaces that blind trust with policy: every device is identified, checked for spoofing and BadUSB behaviour, allowed or blocked within a second, and recorded in an audit trail, while file activity on approved drives is logged and abnormal transfers are flagged. The full source code, logs, screenshots and report are available at github.com/anjalicheriath003/usb-device-control-monitoring.